

Universitatea Națională de Știință și Tehnologie POLITEHNICA București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

Căutarea fuzzy a textului prin algoritmul Damerau-Levenshtein

Profesor
Elena-Cristina Stoica

Student
Asan Aura-Ingrid
Turcescu Mihail

Anul 2026

Cuprins

Lista figurilor	4
Lista tabelelor	6
Lista acronimelor	8
1. Studiu teoretic	10
1.1. Introducere în căutarea fuzzy	10
1.2. Algoritmul Damerau-Levenshtein: Definiție și caracteristici	10
1.3. Formularizare matematică și pseudocod	10
1.4. Domenii de aplicabilitate	11
2. Proiectare	13
2.1. Arhitectura sistemului și strategia de paralelizare	13
2.2. Managementul partițiilor de date și tehnica de suprapunere	13
2.3. Analiza dependențelor, a comunicației și a sincronizării	13
2.4. Tehnologia utilizată: Python și framework-ul gRPC	14
3. Implementare	15
3.1. Suportul de dezvoltare a aplicației	15
3.2. Integrarea și configurarea tehnologiei gRPC	15
3.3. Secvențe relevante la nivelul implementării algoritmice	16
3.4. Validarea funcțională a sistemului	16
4. Rezultate experimentale	18
4.1. Modalitatea și instrumentele de măsurare a timpului de execuție	18
4.2. Analiza și interpretarea rezultatelor obținute	18

Lista figurilor

3.1. Inițializarea nodului Worker și așteptarea conexiunilor gRPC.	16
3.2. Stabilirea canalelor de comunicație de către nodul Master.	17
3.3. Afișarea raportului de execuție și a metricilor de performanță în terminalul clientului.	17

Lista tabelelor

4.1.	Performanța execuției pentru fișierul <i>lorem-ipsa</i> <i>sum.txt</i> în funcție de gradul de similaritate (Pattern: "magna", Chunk Size: 1024, Overlap: 50).	19
4.2.	Performanța execuției pentru fișierul <i>lorem-ipsa</i> <i>sum.txt</i> (Pattern: "magna", Similaritate: 0.5, Overlap: 50).	19
4.3.	Performanța execuției pentru fișierul <i>10MB.txt</i> (Pattern: "he said", Similaritate: 1.0, Overlap: 50).	19

Lista acronimelor

gRPC Google Remote Procedure Call

I/O Input/Output

JSON JavaScript Object Notation

LAN Local Area Network

NLP Natural Language Processing

RPC Remote Procedure Call

TCP/IP Transmission Control Protocol/Internet Protocol

XML Extensible Markup Language

Capitolul 1

Studiu teoretic

1.1 Introducere în căutarea fuzzy

Căutarea aproximativă (cunoscută și sub denumirea de fuzzy string searching) reprezintă o tehnică de identificare a șirurilor de caractere care prezintă un grad definit de similitudine cu un șablon dat, permițând returnarea unor rezultate pozitive chiar și în prezența erorilor sau a variațiilor de formă. Eficiența acestei metode depinde de metrica de distanță aleasă pentru a cuantifica diferențele dintre secvențe.

1.2 Algoritmul Damerau-Levenshtein: Definiție și caracteristici

Distanța Damerau-Levenshtein este o metrică utilizată în teoria informației și informatică pentru a măsura diferența dintre două secvențe de caractere. Această distanță este definită ca fiind numărul minim de operații necesare pentru a transforma un șir sursă într-un șir destinație [1].

Spre deosebire de distanța Levenshtein clasică, care permite doar trei tipuri de modificări, algoritmul Damerau-Levenshtein introduce o a patra operație, generalizând astfel procesul. Setul complet de operații primitive permise include:

- Inserarea unui singur caracter;
- Ștergerea unui singur caracter;
- Substituția unui caracter cu un altul;
- Transpoziția a două caractere adiacente [2].

Câteva exemple concrete de calcul al distanței se regăsesc mai jos:

- Cuvântul "ball" transformat în "bal" necesită o singură operație de ștergere, rezultând o distanță egală cu 1.
- Transformarea șirului "CA" în "ABC" sub incidența acestui algoritm necesită două operații secvențiale: o transpoziție ("CA" devine "AC"), urmată de o inserție ("AC" devine "ABC"), distanța totală fiind 2 [3].

1.3 Formularizare matematică și pseudocod

Distanța dintre două șiruri a și b poate fi definită recursiv printr-o funcție $d_{a,b}(i, j)$, unde parametrii i și j reprezintă lungimea prefixelor evaluate pentru șirul a , respectiv șirul b .

Pentru a optimiza timpul de execuție, algoritmul este implementat uzual prin metoda programării dinamice, utilizând o matrice de costuri. Dacă notăm costul curent cu $cost$ (unde $cost = 0$ în cazul în care caracterele analizate sunt identice și $cost = 1$ în cazul unei

neconcordanțe), ecuația de recurență principală pentru celula $d[i, j]$ din matrice determină minimul dintre următoarele operații [1]:

$$d[i, j] = \min(d[i - 1, j] + 1, d[i, j - 1] + 1, d[i - 1, j - 1] + cost) \quad (1.1)$$

Diferența fundamentală față de algoritmul Levenshtein constă în adăugarea unei recurențe suplimentare pentru tratarea transpozițiilor. Dacă $i > 1$, $j > 1$, $a[i] = b[j - 1]$ și $a[i - 1] = b[j]$, valoarea celulei este actualizată astfel [1]:

$$d[i, j] = \min(d[i, j], d[i - 2, j - 2] + cost) \quad (1.2)$$

Pseudocodul aferent calculului matriceal este detaliat mai jos.

Data: Șirul a de lungime N , Șirul b de lungime M

Result: Distanța Damerau-Levenshtein dintre a și b

Inițializează matricea d de dimensiune $(N + 1) \times (M + 1)$

for $i \leftarrow 0$ **to** N **do**

$d[i, 0] \leftarrow i$

end

for $j \leftarrow 0$ **to** M **do**

$d[0, j] \leftarrow j$

end

for $i \leftarrow 1$ **to** N **do**

for $j \leftarrow 1$ **to** M **do**

if $a[i] == b[j]$ **then**

$cost \leftarrow 0$

else

$cost \leftarrow 1$

end

$d[i, j] \leftarrow \min(d[i - 1, j] + 1, d[i, j - 1] + 1, d[i - 1, j - 1] + cost)$

if $i > 1$ **and** $j > 1$ **and** $a[i] == b[j - 1]$ **and** $a[i - 1] == b[j]$ **then**

$d[i, j] \leftarrow \min(d[i, j], d[i - 2, j - 2] + cost)$

end

end

end

return $d[N, M]$

Algorithm 1: Calculul distanței Damerau-Levenshtein

1.4 Domenii de aplicabilitate

Datorită capacității de a identifica variații comune la nivel de caracter, algoritmul Damerau-Levenshtein este integrat într-o serie de domenii computaționale și de analiză a datelor:

- Procesarea limbajului natural (NLP) și corectarea ortografică: Frederick J. Damerau a observat în cercetările sale că peste 80% din erorile umane de redactare reprezintă o singură modificare de tip inserție, ștergere, substituție sau transpoziție. În limbajul natural, unde numărul de greșeli dintr-un cuvânt este redus, algoritmul este utilizat pentru sugerarea corecturilor.
- Bioinformatică: Algoritmul servește drept metrică pentru evaluarea variațiilor dintre catenele de ADN sau secvențele de proteine. Mutațiile naturale precum inserțiile, ștergerile,

substituțiile sau transpozițiile au loc pe scale de timp similare, făcând această abordare relevantă din punct de vedere biologic.

- Rezolvarea entităților și deduplicarea datelor: Metrica este aplicată în sistemele de baze de date pentru a corela înregistrări (de exemplu, nume de furnizori, coduri poștale sau adrese de e-mail) care prezintă discrepanțe cauzate de erori de introducere a datelor.
- Detectarea fraudelor: Este utilizat în domeniul securității financiare pentru a detecta conturi bancare sau profiluri de utilizatori false, create prin alterarea minimă (cum ar fi inversarea unor litere adiacente) a unor nume reale de furnizori sau clienți [1].

Capitolul 2

Proiectare

2.1 Arhitectura sistemului și strategia de paralelizare

Pentru rezolvarea problemei de căutare a unui șablon într-un volum mare de date (de ordinul gigaoctetelor), arhitectura sistemului se bazează pe paradigma paralelismului de date (Data Parallelism). Această abordare presupune distribuirea setului de date către multiple unități de procesare care execută simultan aceleași instrucțiuni pe submulțimi diferite de date [4].

Sistemul implementează un model de tip Master-Worker. Nodul central (Master) are responsabilitatea de a citi fișierul de intrare, de a-l fragmenta și de a dispecheriza sarcinile de calcul. Nodurile de execuție (Workers) primesc datele, aplică algoritmul Damerau-Levenshtein și returnează rezultatele către nodul inițiator.

2.2 Managementul partițiilor de date și tehnica de suprapunere

Fragmentarea fișierului principal se realizează în blocuri de dimensiuni predefinite (chunk-uri). O provocare inerentă divizării textului pentru căutarea șirurilor de caractere este gestionarea instanțelor în care fraza căutată se află poziționată exact la granița dintre două partiții. Într-o partiționare strictă, o astfel de instanță ar fi trunchiată și, în consecință, omisă de ambii workeri care procesează blocurile adiacente.

Pentru a asigura corectitudinea rezultatelor, se utilizează o strategie de suprapunere a blocurilor de date (overlap) [5]. Fiecare partiție generată de Master, cu excepția primei, include o secvență de caractere de la finalul partiției anterioare.

Dimensiunea parametrului overlap size trebuie calculată matematic pentru a garanta detecția. Pentru o căutare corectă, dimensiunea zonei de suprapunere trebuie să fie strict mai mare sau egală cu suma dintre lungimea șablonului căutat și distanța maximă de editare admisă (gradul de similaritate).

2.3 Analiza dependențelor, a comunicației și a sincronizării

La nivelul algoritmului distribuit, procesul de căutare pe o partiție izolată reprezintă o problemă de tip "perfectly parallel". Nu există dependențe de date între nodurile Worker; execuția pe un nod nu necesită informații de stare sau rezultate parțiale de la un alt nod [4]. Din acest motiv, nu este necesară implementarea unor mecanisme de comunicare inter-worker.

Sincronizarea este centralizată exclusiv la nivelul nodului Master. Fluxul de comunicație urmează un tipar bidirecțional simplu: cerere (Master → Worker) și răspuns (Worker → Master). Deoarece procesarea partițiilor are loc asincron, Masterul necesită un mecanism de agregare a rezultatelor. După primirea pozițiilor (indicilor) de la Workeri, Masterul aplică o funcție de translație, adunând la fiecare index local offset-ul (deplasamentul) partiției respective, pentru a obține poziția globală validă în raport cu fișierul original.

2.4 Tehnologia utilizată: Python și framework-ul gRPC

Sistemul este implementat utilizând limbajul Python, un mediu propice pentru orchestrarea proceselor de I/O, susținut de framework-ul gRPC (Google Remote Procedure Call) pentru comunicația distribuită.

gRPC este un sistem open-source de apelare a procedurilor la distanță care permite conectarea transparentă a serviciilor distribuite [6]. Prin intermediul acestuia, nodul Master poate apela funcția de căutare implementată pe nodurile Worker ca și cum ar fi o funcție locală.

Pentru definirea interfeței și serializarea datelor se utilizează Protocol Buffers (Protobuf). Spre deosebire de protocoalele textuale (precum JSON sau XML), Protobuf compilează datele într-un format binar compact, reducând semnificativ consumul de lățime de bandă și overhead-ul de procesare la serializare/deserializare [7].

Structura comunicației este definită printr-un fișier `.proto` ce conține:

- Mesajul de cerere (Request): Încapsulează partiția de text (string), șablonul căutat (string) și distanța Damerau-Levenshtein maxim admisă (integer).
- Mesajul de răspuns (Response): Returnează un vector sau o listă de numere întregi, reprezentând pozițiile de start ale potrivirilor găsite în blocul de text curent.
- Serviciul RPC: Definește metoda invocabilă de la distanță care primește cererea și emite răspunsul.

Capitolul 3

Implementare

3.1 Suportul de dezvoltare a aplicației

Implementarea sistemului distribuit necesită un mediu de dezvoltare configurat corespunzător pentru a susține execuția concurentă și comunicarea în rețea.

Resurse Hardware: Pentru dezvoltarea și testarea aplicației, este necesară o infrastructură care să permită instanțierea mai multor procese. Un mediu tipic de testare include:

- Procesor (CPU): O arhitectură multi-core (de exemplu, un procesor cu minim 4 nuclee fizice) pentru a permite execuția fizic paralelă a instanțelor Worker locale.
- Memorie (RAM): O capacitate suficientă (ex. 8-16 GB) pentru a susține încărcarea în memorie a partițiilor de text (chunk-urilor) de către nodul Master și procesarea simultană a acestora de către Workeri.
- Rețea: O interfață de rețea configurată pentru stiva TCP/IP. Pentru testarea pe o singură mașină fizică, se utilizează interfața de loopback (localhost). Pentru execuția distribuită propriu-zisă, este necesară o rețea locală (LAN) capabilă să susțină transferul rapid al datelor partiționate

Resurse Software:

- Sistem de operare: Soluția poate fi rulată pe sisteme de tip UNIX/Linux sau Windows, datorită portabilității limbajului ales.
- Tehnologii și librării: Implementarea utilizează limbajul de programare Python (versiunea 3.x). Dependențele externe esențiale sunt pachetele grpcio (care oferă suportul de rulare pentru gRPC) și grpcio-tools (care include compilatorul de protocol buffers) [6].

3.2 Integrarea și configurarea tehnologiei gRPC

Procesul de configurare a infrastructurii de comunicație se bazează pe fișierul de definire a serviciului (`.proto`). Primul pas în implementare îl constituie generarea codului sursă necesar pentru client (Master) și server (Worker).

Acest proces se realizează prin intermediul compilatorului `protoc`, apelat din mediul Python:

```
python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. service.proto.
```

Această comandă generează două module esențiale:

- Modulul care definește structurile de date (clasele pentru Request și Response).
- Modulul care conține clasele "Stub" (utilizate de Master pentru a apela procedurile la distanță) și "Servicer" (extinse de Workeri pentru a implementa logica de calcul) [8].

Configurarea rețelei presupune instanțierea serverelor gRPC pe nodurile Worker, alocându-le adrese și porturi de ascultare specifice (de exemplu, `[::]:50051`, `[::]:50052`). Nodul Master este configurat pentru a deschide canale de comunicație nesecurizate (`grpc.insecure_channel`) către aceste adrese, inițiind conexiunile necesare transferului de date.

3.3 Secvențe relevante la nivelul implementării algoritmice

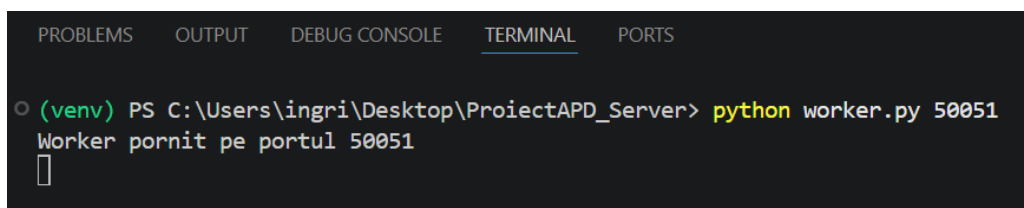
Implementarea practică poate fi divizată în două componente majore, fiecare având funcționalități specifice:

Logica nodului Master: Secvența esențială la nivelul Masterului gestionează divizarea fișierului de intrare și maparea globală a rezultatelor. Citirea fișierului se face iterativ, extrăgând blocuri de date de dimensiunea parametrizată `chunk_size`. La fiecare bloc (cu excepția primului), se adaugă un prefix reprezentând zona de `overlap`, citită de la finalul blocului anterior. Masterul lansează apeluri asincrone către Workeri, atașând un identificator sau un index de deplasament (offset) fiecărui apel, pentru a recalcula pozițiile finale primite: $Poziție_{globala} = Offset_{chunk} + Poziție_{locala} - Dimensiune_{overlap}$.

Logica nodului Worker: Funcția relevantă pe nodul Worker este suprascrierea metodei definite în interfața `Servicer`. La primirea unui mesaj de tip `Request`, Workerul extrage textul, șablonul și limita superioară a distanței Damerau-Levenshtein. Logica implementează o fereastră glisantă (sliding window) care parcurge textul blocului primit, aplicând funcția Damerau-Levenshtein definită anterior asupra ferestrei curente. Dacă distanța calculată este mai mică sau egală cu pragul admis, poziția locală este înregistrată în lista de rezultate.

3.4 Validarea funcțională a sistemului

Pentru a demonstra viabilitatea arhitecturii distribuite, prima etapă a execuției o constituie inițializarea nodurilor de calcul. Figura 3.1 ilustrează terminalul aferent unui nod Worker în faza de pornire. Se observă alocarea portului de ascultare și deschiderea socket-ului de rețea, nodul intrând într-o stare de așteptare pasivă (listening) pentru cererile de tip Remote Procedure Call (RPC) venite din partea nodului central. Această etapă confirmă instanțierea corectă a claselor de tip `Servicer` generate de framework-ul gRPC.



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(venv) PS C:\Users\ingri\Desktop\ProiectAPD_Server> python worker.py 50051
Worker pornit pe portul 50051
█
```

Figura 3.1: Inițializarea nodului Worker și așteptarea conexiunilor gRPC.

Ulterior activării nodurilor de execuție, nodul Master (Clientul) inițiază topologia de rețea. Figura 3.2 prezintă jurnalul de execuție al clientului în momentul stabilirii canalelor de comunicație (channels). Așa cum reiese din captura de ecran, Masterul identifică adresele IP și porturile workerilor disponibili și validează conexiunile înainte de a începe dispecerizarea partițiilor de date (chunk-urilor). Această fază validează funcționalitatea de rețea și absența blocajelor la nivel de firewall sau de rutare locală.

```

D (venv) PS C:\Users\ingri\Desktop\ProiectAPD_Server> python server.py localhost:50051 192.168.1.6:50052
Workeri activi:
- localhost:50051
- 192.168.1.6:50052
Server pornit pe portul 50050

Client conectat: ipv4:192.168.1.6:55225

Client conectat: ipv4:192.168.1.6:55225
Proces inceput
Proces terminat

```

Figura 3.2: Stabilirea canalelor de comunicație de către nodul Master.

Finalizarea procesului de căutare distribuită este marcată de agregarea rezultatelor parțiale. Figura 3.3 expune raportul final afișat în terminalul clientului după recepționarea tuturor răspunsurilor asincrone. Rezultatele atestă procesarea completă a fișierului de intrare, afișând parametrii configurației curente, metricile de performanță înregistrate (timpul total, factorul de accelerare și throughput-ul) și, cel mai important, validarea corectitudinii algoritmice prin numărul total de apariții identificate pentru șablonul căutat.

```

D (venv) PS C:\Users\Ingrid\Desktop\ProiectAPD_client> python client.py
Client pornit
Conectare la server: 192.168.1.4:50050

Fișiere disponibile pe server:

1. 10MB.txt
2. lorem_ipsum.txt
3. text_mare.txt

Scrie exit pentru iesire.

Alege numarul fișierului: 2
Introdu cuvantul/fraza de cautat: lorem
Introdu gradul de similaritate (0-1): 0.8
Chunk size [default 1024]: 1024
Overlap size [default 50]: 30

Trimit cererea catre server...
Pozitii gasite:
[0, 200, 649, 883, 2062, 2296, 3065, 3253, 3748, 4738, 5169, 5385, 6344, 6575, 6744, 11475, 11506, 11592, 13721, 15389, 15426, 15942, 16884, 1736
8, 19260, 20548, 21632, 23074, 24038, 24402, 25552, 26419, 26852, 29416, 30668, 32183, 32338, 33140, 34856, 39616, 41464, 42349, 42540, 43369, 43
742, 44401, 44801, 44852, 46063, 47250, 49674, 50713, 51215, 52242, 52832, 54163, 54397, 55736, 58740, 61006, 61403, 62704, 63946, 64592, 65563, 6
5685, 69002, 69084, 70097, 71793, 71877, 72318, 73583, 73631, 74201, 75271, 76423, 77538, 78002, 79992, 80258, 81473, 82232, 84771, 84854, 87665
, 88210]

===== REZULTATE =====
Fișier: lorem_ipsum.txt
Pattern cautat: lorem
Similaritate: 0.8
Chunk size: 1024
Overlap size: 30
Numar workeri: 2
-----
Timp nedistribuit: 0.565744 secunde
Timp distribuit: 0.578011 secunde
Speedup: 0.98x
-----
Throughput nedistribuit: 156392.18 caractere/secunda
Throughput distribuit: 153073.20 caractere/secunda
-----
Aparitii nedistribuit: 87
Aparitii distribuit: 87
=====

```

Figura 3.3: Afișarea raportului de execuție și a metricilor de performanță în terminalul clientului.

Capitolul 4

Rezultate experimentale

4.1 Modalitatea și instrumentele de măsurare a timpului de execuție

Evaluarea performanței sistemului s-a realizat prin monitorizarea a două tipuri de execuții pentru același set de date: execuția secvențială (nedistribuită) și execuția distribuită utilizând framework-ul gRPC. Pentru a asigura corectitudinea și trasabilitatea datelor, au fost înregistrați și comparați următorii parametri de performanță:

- Timpul nedistribuit (secunde): Timpul total necesar procesării întregului fișier de către un singur proces, fără divizarea datelor sau comunicare în rețea.
- Timpul distribuit (secunde): Timpul total scurs de la inițierea partiționării fișierului de către nodul Master până la primirea și agregarea tuturor rezultatelor de la nodurile Worker.
- Factorul de accelerare (Speedup): Raportul dintre timpul nedistribuit și timpul distribuit, indicând eficiența paralelizării.
- Throughput (caractere/secundă): Volumul de date procesate pe unitatea de timp, calculat atât pentru mediul nedistribuit, cât și pentru cel distribuit.
- Numărul de apariții: Variabilă de control utilizată pentru a valida corectitudinea execuției; numărul de potriviri identificate trebuie să fie identic în ambele scenarii.

Testele au fost rulate pe două seturi distincte de date: un fișier de dimensiuni mici (`lorem_ipsum.txt`) și un fișier extins (`10MB.txt`), variind progresiv dimensiunea partițiilor (Chunk size) între 1024 și 819200 caractere, precum și numărul de workeri (1 și 2).

4.2 Analiza și interpretarea rezultatelor obținute

Datele experimentale au fost centralizate pentru a observa influența granularii datelor și a scalării orizontale asupra performanței generale.

Tabel 4.1: Performanța execuției pentru fișierul *lorem-ipsa.txt* în funcție de gradul de similaritate (Pattern: "magna", Chunk Size: 1024, Overlap: 50).

Workeri	Similaritate	Timp Nedistribuit (s)	Timp Distribuit (s)	Speedup	Throughput Nedistribuit (char/s)	Throughput Distribuit (char/s)	Apariții
1	0.0	0.538619	0.658921	0.82x	164268.24	134277.10	88478
1	0.5	4.895312	5.226153	0.94x	18074.03	16929.85	525
1	0.8	0.437210	0.526653	0.83x	202369.53	168000.47	81
1	1.0	0.432250	0.512930	0.84x	204691.49	172495.14	81
2	0.0	0.802578	1.757030	0.46x	110242.22	50356.57	88478
2	0.5	6.542582	4.580248	1.43x	13523.41	19317.29	525
2	0.8	0.652757	0.728850	0.90x	135545.14	121394.04	81
2	1.0	0.902612	1.261907	0.72x	98024.40	70114.52	81

Tabel 4.2: Performanța execuției pentru fișierul *lorem-ipsa.txt* (Pattern: "magna", Similaritate: 0.5, Overlap: 50).

Workeri	Chunk Size	Timp Nedistribuit (s)	Timp Distribuit (s)	Speedup	Throughput Nedistribuit (char/s)	Throughput Distribuit (char/s)
1	1024	4.895312	5.226153	0.94x	18074.03	16929.85
2	1024	6.542582	4.580248	1.43x	13523.41	19317.29
1	2048	5.323907	5.167220	1.03x	16619.00	17122.94
2	2048	7.340303	3.904474	1.88x	12053.73	22660.67
1	8192	5.098169	4.977596	1.02x	17354.86	17775.25
2	8192	10.555477	5.674515	1.86x	8382.19	15592.17

Tabel 4.3: Performanța execuției pentru fișierul *10MB.txt* (Pattern: "he said", Similaritate: 1.0, Overlap: 50).

Workeri	Chunk Size	Timp Nedistribuit (s)	Timp Distribuit (s)	Speedup	Throughput Nedistribuit (char/s)	Throughput Distribuit (char/s)
1	1024	76.493386	87.227909	0.88x	139831.28	122623.23
2	1024	97.572265	238.606252	0.41x	109623.04	44827.69
1	4096	80.177634	76.077641	1.05x	133405.88	140595.42
2	4096	97.292284	100.405783	0.97x	109938.50	106529.40
1	102400	71.469128	69.231501	1.03x	149661.37	154498.57
2	102400	98.202268	53.921944	1.82x	108919.77	198363.92
1	819200	75.088527	76.484541	0.98x	142447.43	139847.45
2	819200	96.218610	51.181710	1.88x	111165.27	208984.19

Pe baza rezultatelor obținute mai sus, s-au făcut următoarele interpretări, punctând 4 aspecte principale:

1. **Impactul gradului de similaritate asupra complexității de calcul:** Datele agregate în Tabelul 4.1 evidențiază influența directă a parametrului de similaritate atât asupra volumului de rezultate, cât și asupra efortului computațional. Relaxarea pragului de acceptare (reducerea similarității de la 1.0 la 0.5) determină o creștere justificată a numărului de potriviri (de la 81 la 525 apariții), însă impune o penalizare temporală severă. Pe configurația cu un singur worker, timpul distribuit crește cu un ordin de mărime, de la 0.51 secunde (similaritate 1.0) la 5.22 secunde (similaritate 0.5). Acest fenomen se explică prin necesitatea parcurgerii exhaustive a matricei de costuri în algoritmul Damerau-Levenshtein, întrucât permisiunea unui număr mai mare de erori previne respingerea timpurie a ramurilor de calcul. Cazul limită, cu o similaritate de 0.0, generează un număr masiv de false pozitive (88478 apariții) într-un timp redus de execuție (0.65 secunde), indicând o validare imediată la nivel algoritmic a oricărei secvențe din text, fără un calcul efectiv al distanței.
2. **Impactul dimensiunii partiției (Chunk Size) și a latenței de rețea:** Analiza Tabelului 4.3 relevă un comportament clasic al sistemelor distribuite supuse unei fragmentări excesive a datelor. Pentru fișierul de 10 MB, utilizarea unui Chunk Size de 1024 caractere cu 2 workeri generează un timp distribuit masiv (238.60 secunde) și un Speedup subunitar critic de 0.41x. Acest fenomen indică faptul că volumul operațiunilor de serializare Protobuf și stabilirea conexiunilor gRPC (overhead-ul de comunicare) depășesc cu mult timpul efectiv necesar calculului distanței Damerau-Levenshtein. Odată cu creșterea dimensiunii partiției la 102400 și 819200 caractere, timpul de comunicare devine neglijabil în raport cu timpul de procesare. Astfel, pentru un Chunk Size de 819200 caractere, timpul distribuit pe 2 workeri scade la 51.18 secunde, atingând un Speedup optim de 1.88x și un throughput maxim de peste 208000 caractere/secundă.
3. **Eficiența paralelizării și scalabilitatea:** Adăugarea unui al doilea worker demonstrează o scalabilitate liniară pozitivă, dar strict dependentă de un Chunk Size calibrat corect. În cazul fișierului `lorem_ipsum.txt` cu Chunk Size de 2048, trecerea de la 1 la 2 workeri îmbunătățește Speedup-ul de la 1.03x la 1.88x. Valoarea apropiată de pragul teoretic ideal de 2.0x confirmă faptul că arhitectura gRPC implementată distribuie uniform sarcina de calcul între noduri.
4. **Influența sistemului de operare și variațiile mediului de execuție:** Se observă fluctuații semnificative în măsurarea timpului nedistribuit între rulări succesive, în special în Tabelul 4.2, unde valorile variază de la 6.54 secunde la 10.55 secunde pentru aceleași date de intrare. Aceste deviații sunt caracteristice mediilor de testare non-dedicate și sunt cauzate de mecanismele sistemului de operare gazdă (planificarea proceselor, context switching) și de alocarea dinamică a resurselor CPU către alte aplicații terțe în timpul execuției secvențiale.

Sistemul paralel-distribuit devine net superior implementării secvențiale doar atunci când raportul dintre volumul de calcul per sarcină și latența rețelei este ridicat. Testele validează necesitatea utilizării partițiilor mari (de ordinul sutelor de kilobytes sau megabytes) pentru a amortiza costul operațiunilor RPC, scenariu în care arhitectura Master-Worker implementată reușește să înjumătățească timpii de procesare comparativ cu rularea pe un singur proces.

Bibliografie

- [1] https://en.wikipedia.org/wiki/Damerau-Levenshtein_distance.
- [2] Canada University of Waterloo. Damerau-levenstein distance. https://ece.uwaterloo.ca/~dwharder/aads/Algorithms/String_distances/Damerau-Levenshtein/.
- [3] <https://mathspp.com/blog/til/damerau-levenshtein-distance>.
- [4] Dollimore J. Kindberg T. Blair G. Coulouris, G. *Distributed Systems: Concepts and Design (5th Edition)*. Addison-Wesley, 2011.
- [5] G. Navarro. A guided tour to approximate string matching. *ACM Computing Surveys (CSUR)*, 33(1):31–88, 2001.
- [6] https://en.wikipedia.org/wiki/Damerau-Levenshtein_distance.
- [7] S. Vinoski. Advanced message queuing protocol. *IEEE Internet Computing*, 10(6):87–89, 2006.